

1. You are given a directed graph $G = (V, E)$. A vertex v is called nice if there is a directed path from v to each of the other vertices in G . Suppose we run DFS on G starting from some vertex s and let w be the vertex with the highest finish time $F[w]$. Prove that if G has a nice vertex, then w is a nice vertex.

↳ $G = (V, E)$

v is nice iff $\exists \rightarrow u \quad \forall u \in V \text{ and } u \neq v$

Start vertex s

$F(w)$ highest finish time, i.e. w is last to finish.

↳ If a vertex can reach every other vertex then that means when we call DFS then it will end up calling every other vertex and end at last. Let's assume nice vertex is some u

Case (1) we visit u from some vertex x .



↳ now as u connects to every other vertex whose $\text{visited}[u] = \text{false}$ we visit those vertex from u and hence u finishes last i.e. maximum $F[u]$

↳ property of w



↳ All remaining nodes

Case (2) u is not visited by any other node

↳ u has no incoming edge,

Case (2.a)

↳ some vertex are remaining, similar as case (1)

Case (2.b)

↳ u is last to visit, as every other vertex is visited and finished u will have the highest finish time as no vertex called DFS(u).

↳ from both cases we can conclude that if u is a nice vertex then it finishes last.

2. Let G be a directed graph. Let $C_1, C_2, \dots, C_k$ denote the strongly connected components of G (each C_i is a subset of vertices). We now construct another directed graph H as follows: there are k vertices in H , denoted $v_1, \dots, v_k$. There is a directed edge (v_i, v_j) in H if and only if there is an edge in G from a vertex in C_i to a vertex in C_j . Prove that H is a DAG. Give a linear time algorithm to construct H .

↳ $C_i \subset V \quad G = (V, E)$

$(v_i, v_j) \in H$ iff $v_i \in C_i$ and $v_j \in C_j$ and $(u_i, v_j) \in E$

to prove H is DAG.

↳ assume H is not DAG, i.e. it has cycle.

?

3. You are given a directed acyclic graph. Each edge e has a length ℓ_e . The length of a path is the total length of the edges in it. Given a vertex s , give a linear time algorithm to find the longest path that starts from s .

Components

- ① global array $visited[v] = \text{false}$ for $v \in V$
- ② global dictionary holding edge length. $Edge[u, v]$ - edge length from u to v
- ③ max-D global array. keep track of distance from s to u initialized to $-\infty$

DFS(u) &

if $max-D[u] \neq -\infty$:
return $max-D[u]$

$max-distance = 0$

for all neighbours v of u :

if $visited[v]$ is false:

current-distance $Edge[u, v] + DFS(v)$

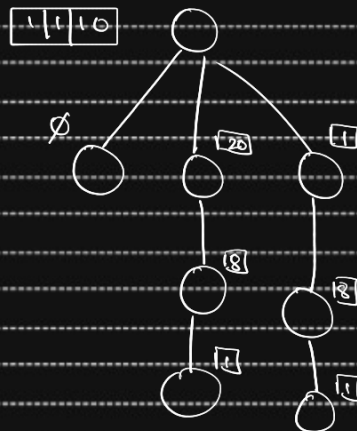
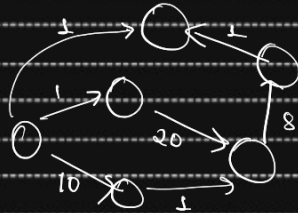
if current-distance $> max-distance$:

$max-distance = \text{current-distance}$

$max-D[u] = max-distance$

return $max-distance$

4



4. A bipartite graph is an undirected graph whose vertex set can be partitioned into two sets A and B such that each edge in the graph goes between a vertex in A and a vertex in B (no edges exist between vertices in the same set). Give an $O(|V| + |E|)$ time algorithm that takes an input graph represented as an adjacency list, and decides if the graph is bipartite.

Components

① global array $visited[v] = \text{false}$ for $v \in V$

② global array $side[v] = -1$ for $v \in V$
-1 not

DFS(u) &

$visited[u] = \text{true}$

for all neighbours v of u :

case ① neighbour not visited:

if $visited[v]$ is false:

$DFS(v)$

case ② neighbour visited

else

if $side[v] \neq 0$: return not bipartite

else: let it be.

5

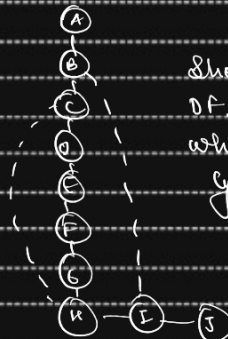
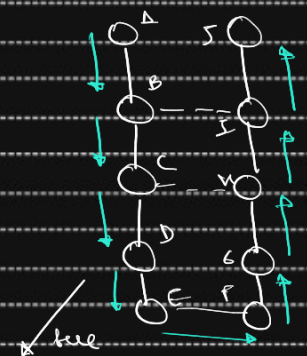
5. You are given an undirected graph G in the adjacency list data-structure. We would like to find the shortest cycle in the graph (length of a cycle is defined as the number of edges in it).

- You try the following DFS based algorithm: construct the DFS tree (assume it is connected, otherwise we can run this on each connected component). For each back edge, find the length of the corresponding cycle and report the shortest cycle. Give an example where this algorithm fails to find the shortest cycle.
- Give a BFS based algorithm to find the shortest cycle. What is the running time of your algorithm?

1



DFS



Shortest cycle according to DFS = $DDEF$ of length 4 while actual shortest cycle $BCHIB$.

② ?

6. Given an undirected graph G and two vertices s and t , we know that BFS can be used to find the shortest path from s to t . We say that there is a *unique* shortest path from s to t if there is exactly one shortest path from s to t . How would you modify BFS to check if there is a unique shortest path from s to t ? The running time of your algorithm should still be linear.

↳ we can count if another node can reach t within same steps.

Components

① Queue Q

② $\text{count}[v] = 0$ for all $v \in V$

③ $\text{dist}[v] = \infty$ for all $v \in V$

↳ same as $\text{visited}[v] = \text{false}$.

BFS $G) \&$

$Q.\text{enqueue}(s)$

$\text{visited}[s] = \text{true}$

repeat till Q is not empty

$x = Q.\text{dequeue}()$

for each neighbour w of x :

if $\text{dist}[w] \text{ is } \infty$:

$\text{dist}[w] = \text{dist}[x] + 1$

$\text{count}[w] = \text{count}[x]$

else if $\text{dist}[w] = \text{dist}[x] + 1$

$\text{count}[w] += \text{count}[x]$ } new route

} Propagate
count of
Parent.

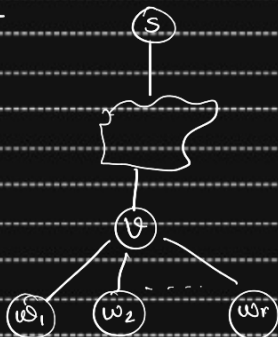
5

Q.7

$G \rightarrow$ Undirected graph

v is cut vertex iff $V' = V - \{v\}$ makes $G' = (V', E')$ disconnected
 $E' = E - \{e(v, u) \mid \text{edge from } v \text{ to } u\}$

$\text{DFS}(s) \rightarrow T$



To show v is not cut vertex iff there is an edge from descendant of w_i to ancestor of v

$\Rightarrow$ Prove, v is not a cut vertex.

Then removing v does not disconnect graph G

i.e. any w_i is still connected to ancestor of v . Therefore there must be an edge from directly from w_i to u or descendant of w_i to u

$\Leftarrow$ Prove, descendant of w_i connects to ancestor of v .

easy to prove